

图形学个人报告

学号：22320021 姓名：陈安康

在图形学大作业中，我主要完成场景障碍物素材的加载和碰撞检测。

场景与障碍物的动态加载生成

障碍物的动态加载

障碍物采用随机生成的方式，每隔一段时间进行概率(obj_factor控制，起始为0.3，随着随着时间的进行，障碍物的生成概率会不断增加)检查，在小球运动轨迹的前方调用绘制函数绘制障碍物，阻碍小球前进，同时为了保证障碍物一直存在导致小球在不停转弯后陷入“死胡同”，减小障碍物绘制增加导致的性能开销，每隔一段时间固定对已经生成的障碍物进行清除。

实现方面，维护一个队列(objs)，存放生成障碍物所需要的位置、时间等信息。在每次渲染时，依据概率进行障碍物添加，删除存放时间过早的障碍物，将存放的参数信息传入绘制函数进行绘制。代码如下：

障碍物的绘制和删除：

```

01. // 生成障碍物
02. obj_count--;
03. float number = dis(gen);
04. if (number < obj_factor && obj_count == 0) {
05.     obj_count = obj_delta;
06.     float x, y, z, size;
07.     if (ball.velocityX != 0) {
08.         y = ball.ballY;
09.         float sign = (ball.velocityX > 0) ? 1.0f : -1.0f;
10.         x = ball.ballX + sign * front;
11.         z = ball.ballZ;
12.         size = obj_size;
13.     }
14.     else {
15.         x = ball.ballX;
16.         float sign = (ball.velocityY > 0) ? 1.0f : -1.0f;
17.         y = ball.ballY + sign * front;
18.         z = ball.ballZ;
19.         size = obj_size;
20.     }
21.     float current = time_ + keep_time;
22.     std::vector<float> tmp = { x,y,z,size,current,float(dis_int2(gen)) };
23.     objs.push_back(tmp);
24.
25. }
26. else if (obj_count == 0) {
27.     obj_count = obj_delta;
28. }
29.
30.
31. while (!objs.empty()) {
32.     if (time_ > (*objs.begin())[4]) {
33.         objs.erase(objs.begin());
34.     }
35.     else {
36.         break;
37.     }
38. }

```

```

01. // 生成障碍物
02. obj_count--;
03. float number = dis(gen);
04. if (number < obj_factor && obj_count == 0) {
05.     obj_count = obj_delta;
06.     float x, y, z, size;
07.     if (ball.velocityX != 0) {
08.         y = ball.ballY;
09.         float sign = (ball.velocityX > 0) ? 1.0f : -1.0f;
10.         x = ball.ballX + sign * front;
11.         z = ball.ballZ;
12.         size = obj_size;
13.     }
14.     else {
15.         x = ball.ballX;
16.         float sign = (ball.velocityY > 0) ? 1.0f : -1.0f;
17.         y = ball.ballY + sign * front;
18.         z = ball.ballZ;
19.         size = obj_size;
20.     }
21.     float current = time_ + keep_time;
22.     std::vector<float> tmp = { x,y,z,size,current,float(dis_int2(gen)) };
23.     objs.push_back(tmp);
24.
25. }
26. else if (obj_count == 0) {

```

```
27.         obj_count = obj_delta;
28.     }
29.
30.
31.     while (!objs.empty()) {
32.         if (time_ > (*objs.begin())[4]) {
33.             objs.erase(objs.begin());
34.         }
35.         else {
36.             break;
37.         }
38.     }
```

障碍物的绘制:

```

01. for (int i = 0; i < objs.size(); ++i) {
02.     if (objs[i][5] == 0) {
03.         // 交通灯
04.         obj.TrafficLight(objs[i][0], objs[i][1], objs[i][2], objs[i][3] * 0.5);
05.         aabb_detect.Add_Channel(
06.             objs[i][0] - objs[i][3] / 4, // min_x
07.             objs[i][1] - objs[i][3] / 4, // min_y
08.             objs[i][2] - 7.3 * objs[i][3] * 0.5 / 2, // min_z (交通灯高度为 7.3, 缩放比例为 objs[i][3] * 0.5)
09.             objs[i][0] + objs[i][3] / 4, // max_x
10.             objs[i][1] + objs[i][3] / 4, // max_y
11.             objs[i][2] + 7.3 * objs[i][3] * 0.5 / 2 // max_z
12.         );
13.     }
14.     else if (objs[i][5] == 1) {
15.         // 箱子
16.         obj.Box(objs[i][0], objs[i][1], objs[i][2], objs[i][3]);
17.         aabb_detect.Add_Channel(
18.             objs[i][0] - objs[i][3] / 2, // min_x
19.             objs[i][1] - objs[i][3] / 2, // min_y
20.             objs[i][2] - objs[i][3] / 2, // min_z
21.             objs[i][0] + objs[i][3] / 2, // max_x
22.             objs[i][1] + objs[i][3] / 2, // max_y
23.             objs[i][2] + objs[i][3] / 2 // max_z
24.         );
25.     }
26.     else if (objs[i][5] == 2) {
27.         // 垃圾桶
28.         obj.TrashCan(objs[i][0], objs[i][1], objs[i][2], objs[i][3] * 0.5);
29.         aabb_detect.Add_Channel(
30.             objs[i][0] - objs[i][3] / 4, // min_x
31.             objs[i][1] - objs[i][3] / 4, // min_y
32.             objs[i][2] - 2.75 * objs[i][3] * 0.5 / 2, // min_z (垃圾桶高度为 2.75, 缩放比例为 objs[i][3] * 0.5)
33.             objs[i][0] + objs[i][3] / 4, // max_x
34.             objs[i][1] + objs[i][3] / 4, // max_y
35.             objs[i][2] + 2.75 * objs[i][3] * 0.5 / 2 // max_z
36.         );
37.     }
38.     else if (objs[i][5] == 3) {
39.         // 锥形路障
40.         obj.ConeRoadblock(objs[i][0], objs[i][1], objs[i][2], objs[i][3] * 0.25);
41.         aabb_detect.Add_Channel(
42.             objs[i][0] - objs[i][3] / 8, // min_x
43.             objs[i][1] - objs[i][3] / 8, // min_y
44.             objs[i][2] - 5.8 * objs[i][3] * 0.25 / 2, // min_z (锥形路障高度为 5.8, 缩放比例为 objs[i][3] * 0.25)
45.             objs[i][0] + objs[i][3] / 8, // max_x
46.             objs[i][1] + objs[i][3] / 8, // max_y
47.             objs[i][2] + 5.8 * objs[i][3] * 0.25 / 2 // max_z
48.         );
49.     }
50. }

01. for (int i = 0; i < objs.size(); ++i) {
02.     if (objs[i][5] == 0) {
03.         // 交通灯
04.         obj.TrafficLight(objs[i][0], objs[i][1], objs[i][2], objs[i][3] * 0.5);
05.         aabb_detect.Add_Channel(
06.             objs[i][0] - objs[i][3] / 4, // min_x
07.             objs[i][1] - objs[i][3] / 4, // min_y
08.             objs[i][2] - 7.3 * objs[i][3] * 0.5 / 2, // min_z (交通灯高度为 7.3, 缩放比例为 objs[i][3] * 0.5)
09.             objs[i][0] + objs[i][3] / 4, // max_x
10.             objs[i][1] + objs[i][3] / 4, // max_y
11.             objs[i][2] + 7.3 * objs[i][3] * 0.5 / 2 // max_z
12.         );
13.     }
14.     else if (objs[i][5] == 1) {
15.         // 箱子
16.         obj.Box(objs[i][0], objs[i][1], objs[i][2], objs[i][3]);
17.         aabb_detect.Add_Channel(
18.             objs[i][0] - objs[i][3] / 2, // min_x
19.             objs[i][1] - objs[i][3] / 2, // min_y
20.             objs[i][2] - objs[i][3] / 2, // min_z
21.             objs[i][0] + objs[i][3] / 2, // max_x
22.             objs[i][1] + objs[i][3] / 2, // max_y
23.             objs[i][2] + objs[i][3] / 2 // max_z
24.         );
25.     }
26.     else if (objs[i][5] == 2) {
27.         // 垃圾桶
28.         obj.TrashCan(objs[i][0], objs[i][1], objs[i][2], objs[i][3] * 0.5);
29.         aabb_detect.Add_Channel(
30.             objs[i][0] - objs[i][3] / 4, // min_x
31.             objs[i][1] - objs[i][3] / 4, // min_y
32.             objs[i][2] - 2.75 * objs[i][3] * 0.5 / 2, // min_z (垃圾桶高度为 2.75, 缩放比例为 objs[i][3] * 0.5)
33.             objs[i][0] + objs[i][3] / 4, // max_x
34.             objs[i][1] + objs[i][3] / 4, // max_y
35.             objs[i][2] + 2.75 * objs[i][3] * 0.5 / 2 // max_z
36.         );
37.     }
38.     else if (objs[i][5] == 3) {
39.         // 锥形路障
40.         obj.ConeRoadblock(objs[i][0], objs[i][1], objs[i][2], objs[i][3] * 0.25);
41.         aabb_detect.Add_Channel(
42.             objs[i][0] - objs[i][3] / 8, // min_x

```



```

43.         objs[i][1] = objs[i][3] / 8, // min_y
44.         objs[i][2] = 5.8 * objs[i][3] * 0.25 / 2, // min_z (锥形路障高度为 5.8, 缩放比例为 objs[i][3] * 0.25)
45.         objs[i][0] + objs[i][3] / 8, // max_x
46.         objs[i][1] + objs[i][3] / 8, // max_y
47.         objs[i][2] + 5.8 * objs[i][3] * 0.25 / 2 // max_z
48.     );
49. }
50. }

```

场景建筑的动态加载

对于场景建筑的加载，大体流程沿用障碍物绘制流程。不过由于场景只起装饰作用，因此需要保证场景不会与小球相撞，同时为了实现性能优化和小球转向导致走进场景建筑的“死胡同”。场景加载是动态生成的。

为了保证场景动态生成的同时不影响视觉体验，设置检查点（check_x和check_y），初始或者方向改变后小球检查点重置为小球位置。规定当前小球行动方向为正方向，以小球越过检查点作为绘制开始的信号。在正方向两侧一定宽度处（edge），沿正方向一定范围（ter_front）开始每隔一段位置（margin）以概率选边绘制建筑物，防止障碍物穿模。同时，由于场景房屋是在小球两边绘制的，当小球转向时，为了避免小球“撞击”房屋模型，需要判断小球当前方向是否发生改变（通过direct数组存储行进速度来实现），当检查到速度方向改变时，需要清除所有的建筑物，并重新设置检查点，这样就可以实现场景的无缝衔接（理论上会有一帧的误差，也可以通过调整代码顺序优化掉，不过由于人的肉眼完全察觉不出来，所以就没有进行优化，效果依然维持）。

代码上，与障碍物的流程相同，同样是维护一个队列存放建筑绘制的信息，传入绘制函数进行绘制。不过添加逻辑不一样，多出了方向判断清除功能。清除代码如下：

```

01. while (!objs.empty()) {
02.     if (time_ > (*objs.begin())[4]) {
03.         objs.erase(objs.begin());
04.     }
05.     else {
06.         break;
07.     }
08. }
09. if (direct[0] != ball.velocityX || direct[1] != ball.velocityY) {
10.     ters.clear();
11.     direct[0] = ball.velocityX;
12.     direct[1] = ball.velocityY;
13.     check_x = ball.ballX;
14.     check_y = ball.ballY;
15. }

01. while (!objs.empty()) {
02.     if (time_ > (*objs.begin())[4]) {
03.         objs.erase(objs.begin());
04.     }
05.     else {
06.         break;
07.     }
08. }
09. if (direct[0] != ball.velocityX || direct[1] != ball.velocityY) {
10.     ters.clear();
11.     direct[0] = ball.velocityX;
12.     direct[1] = ball.velocityY;
13.     check_x = ball.ballX;
14.     check_y = ball.ballY;
15. }

```

添加代码：

```

01. // 地形
02. //ter_count--;
03. if (ters.size() == 0 || abs(check_x - ball.ballX) + abs(check_y - ball.ballY) < check_dis) {
04.     /*ter_count = ter_delta;*/
05.     float x, y, z, size;
06.     if (ball.velocityX != 0) {
07.         float sign = (ball.velocityX > 0) ? 1.0f : -1.0f;
08.         for (float i = check_x; abs(i - check_x) < ter_front; i += sign * margin) {
09.             x = i;
10.             float left_or_right = dis(gen) > 0.5 ? 1.0f : -1.0f;
11.             y = ball.ballY + left_or_right * edge;
12.             z = ball.ballZ;
13.             size = ter_size;
14.             float current = time_ + ter_keep_time;
15.             //std::vector<float> tmp = { x,y,z,size,current,float(dis_int(gen)) };
16.             ters.push_back({ x,y,z,size,current,float(dis_int(gen)) });
17.         }
18.         check_x = check_x + sign * ter_front;
19.
20.     }
21.     else {
22.         float sign = (ball.velocityY > 0) ? 1.0f : -1.0f;
23.         for (float i = check_y; abs(i - check_y) < ter_front; i += sign * margin) {
24.             y = i;
25.             float left_or_right = dis(gen) > 0.5 ? 1.0f : -1.0f;
26.             x = ball.ballX + left_or_right * edge;
27.             z = ball.ballZ;
28.             size = ter_size;
29.             float current = time_ + ter_keep_time;
30.             //std::vector<float> tmp = { x,y,z,size,current,float(dis_int(gen)) };
31.             ters.push_back({ x,y,z,size,current,float(dis_int(gen)) });
32.         }
33.         check_y = check_y + sign * ter_front;
34.     }
35.     //float current = time_ + ter_keep_time;
36. }

01. // 地形
02. //ter_count--;
03. if (ters.size() == 0 || abs(check_x - ball.ballX) + abs(check_y - ball.ballY) < check_dis) {
04.     /*ter_count = ter_delta;*/
05.     float x, y, z, size;
06.     if (ball.velocityX != 0) {
07.         float sign = (ball.velocityX > 0) ? 1.0f : -1.0f;
08.         for (float i = check_x; abs(i - check_x) < ter_front; i += sign * margin) {
09.             x = i;
10.             float left_or_right = dis(gen) > 0.5 ? 1.0f : -1.0f;
11.             y = ball.ballY + left_or_right * edge;
12.             z = ball.ballZ;
13.             size = ter_size;
14.             float current = time_ + ter_keep_time;
15.             //std::vector<float> tmp = { x,y,z,size,current,float(dis_int(gen)) };
16.             ters.push_back({ x,y,z,size,current,float(dis_int(gen)) });
17.         }
18.         check_x = check_x + sign * ter_front;
19.
20.     }
21.     else {
22.         float sign = (ball.velocityY > 0) ? 1.0f : -1.0f;
23.         for (float i = check_y; abs(i - check_y) < ter_front; i += sign * margin) {
24.             y = i;
25.             float left_or_right = dis(gen) > 0.5 ? 1.0f : -1.0f;
26.             x = ball.ballX + left_or_right * edge;
27.             z = ball.ballZ;
28.             size = ter_size;
29.             float current = time_ + ter_keep_time;
30.             //std::vector<float> tmp = { x,y,z,size,current,float(dis_int(gen)) };
31.             ters.push_back({ x,y,z,size,current,float(dis_int(gen)) });
32.         }
33.         check_y = check_y + sign * ter_front;
34.     }
35.     //float current = time_ + ter_keep_time;
36. }

```

碰撞检测

由于小球只能沿与XY轴平行的方向移动，因此使用AABB碰撞检测是再合适不过的了。

AABB (Axis - Aligned Bounding Box) 碰撞检查是一种用于检测两个物体是否发生碰撞的简单而有效的方法。它的基本思想是用一个最小的轴向对齐的矩形（在 2D 中）或长方体（在 3D 中）来包围物体。这个包围盒的边与坐标轴平行，这样就简化了碰撞检测的计算。

当两个 AABB 的投影在所有坐标轴（在 2D 中有两个坐标轴 x 和 y ，在 3D 中有三个坐标轴 x 、 y 和 z ）上都有重叠时，就认为这两个 AABB 所包围的物体发生了碰撞。反之，如果在任何一个坐标轴上它们的投影没有重叠，那么这两个物体就没有碰撞。这是基于分离轴定理（SAT）的一种简化情况，因为 AABB 的边是与坐标轴平行的，所以我们只需要检查坐标轴方向就可以了。

假设我们有两个物体A和B，用 A_{minX} 或者 A_{maxX} 表示A在X方向的最小值和最大值，物体B或者其余维度同理。

在 x 轴方向上： $A_{minX} \leq B_{maxX}$ and $A_{maxX} \geq B_{minX}$

在 y 轴方向上： $A_{minY} \leq B_{maxY}$ and $A_{maxY} \geq B_{minY}$

在 z 轴方向上： $A_{minZ} \leq B_{maxZ}$ and $A_{maxZ} \geq B_{minZ}$

当这三个条件同时满足时，AB物体发生碰撞。

代码实现上，实现一个两个类，AABB包含物体在某个维度的最大最小值，AABB_detect类用于检测。定义如下：

```

01. struct AABB {
02.     float min_x, min_y, min_z; // AABB 的最小坐标
03.     float max_x, max_y, max_z; // AABB 的最大坐标
04. };
05.
06. class AABB_detect {
07. public:
08.
09.     std::vector<struct AABB> Channel;//可供小球通过的通道AABB的定义
10.     struct AABB ball;
11.     float ballz; //用于上跳检测
12.     float obj_size = 2;
13.     // 构造函数, 用于初始化AABB
14.     AABB_detect() {}
15.     // 初始化小球的 AABB
16.     void Init_AABB_ball(float min_x, float min_y, float min_z, float max_x, float max_y, float max_z) {
17.         ball.min_x = min_x;
18.         ball.min_y = min_y;
19.         ball.min_z = min_z;
20.         ball.max_x = max_x;
21.         ball.max_y = max_y;
22.         ball.max_z = max_z;
23.     }
24.     //添加地图中可供小球通过的通道的AABB, 为矩形
25.     void Add_Channel(float min_x, float min_y, float min_z, float max_x, float max_y, float max_z) {
26.         AABB new_channel = { min_x, min_y, min_z, max_x, max_y, max_z };
27.         Channel.push_back(new_channel);
28.     }
29.
30. struct AABB {
31.     float min_x, min_y, min_z; // AABB 的最小坐标
32.     float max_x, max_y, max_z; // AABB 的最大坐标
33. };
34.
35. class AABB_detect {
36. public:
37.
38.     std::vector<struct AABB> Channel;//可供小球通过的通道AABB的定义
39.     struct AABB ball;
40.     float ballz; //用于上跳检测
41.     float obj_size = 2;
42.     // 构造函数, 用于初始化AABB
43.     AABB_detect() {}
44.     // 初始化小球的 AABB
45.     void Init_AABB_ball(float min_x, float min_y, float min_z, float max_x, float max_y, float max_z) {
46.         ball.min_x = min_x;
47.         ball.min_y = min_y;
48.         ball.min_z = min_z;
49.         ball.max_x = max_x;
50.         ball.max_y = max_y;
51.         ball.max_z = max_z;
52.     }
53.     //添加地图中可供小球通过的通道的AABB, 为矩形
54.     void Add_Channel(float min_x, float min_y, float min_z, float max_x, float max_y, float max_z) {
55.         AABB new_channel = { min_x, min_y, min_z, max_x, max_y, max_z };
56.         Channel.push_back(new_channel);
57.     }
58.

```

在AABB_detect里面实现检测算法:


```

01. bool detect_single_Channel(int i) {
02.     // 检查 x 轴是否重叠
03.     if (ball.min_x > Channel[i].max_x || ball.max_x < Channel[i].min_x) {
04.         return false; // x 轴不重叠, 不相交
05.     }
06.     // 检查 y 轴是否重叠
07.     if (ball.min_y > Channel[i].max_y || ball.max_y < Channel[i].min_y) {
08.         return false; // y 轴不重叠, 不相交
09.     }
10.     // 检查 z 轴是否重叠
11.     if (ball.min_z > Channel[i].max_z || ball.max_z < Channel[i].min_z) {
12.         return false; // z 轴不重叠, 不相交
13.     }
14.     return true; // x、y、z 轴都重叠, 相交
15. }
16. bool detect_channel_ball() {
17.     for (int i = 0; i < Channel.size(); i++) {
18.         if (detect_single_Channel(i) == 1) return 1; // 相交出界
19.     }
20.     return 0;
21. }
22. void clear() {
23.     Channel.clear();
24. }

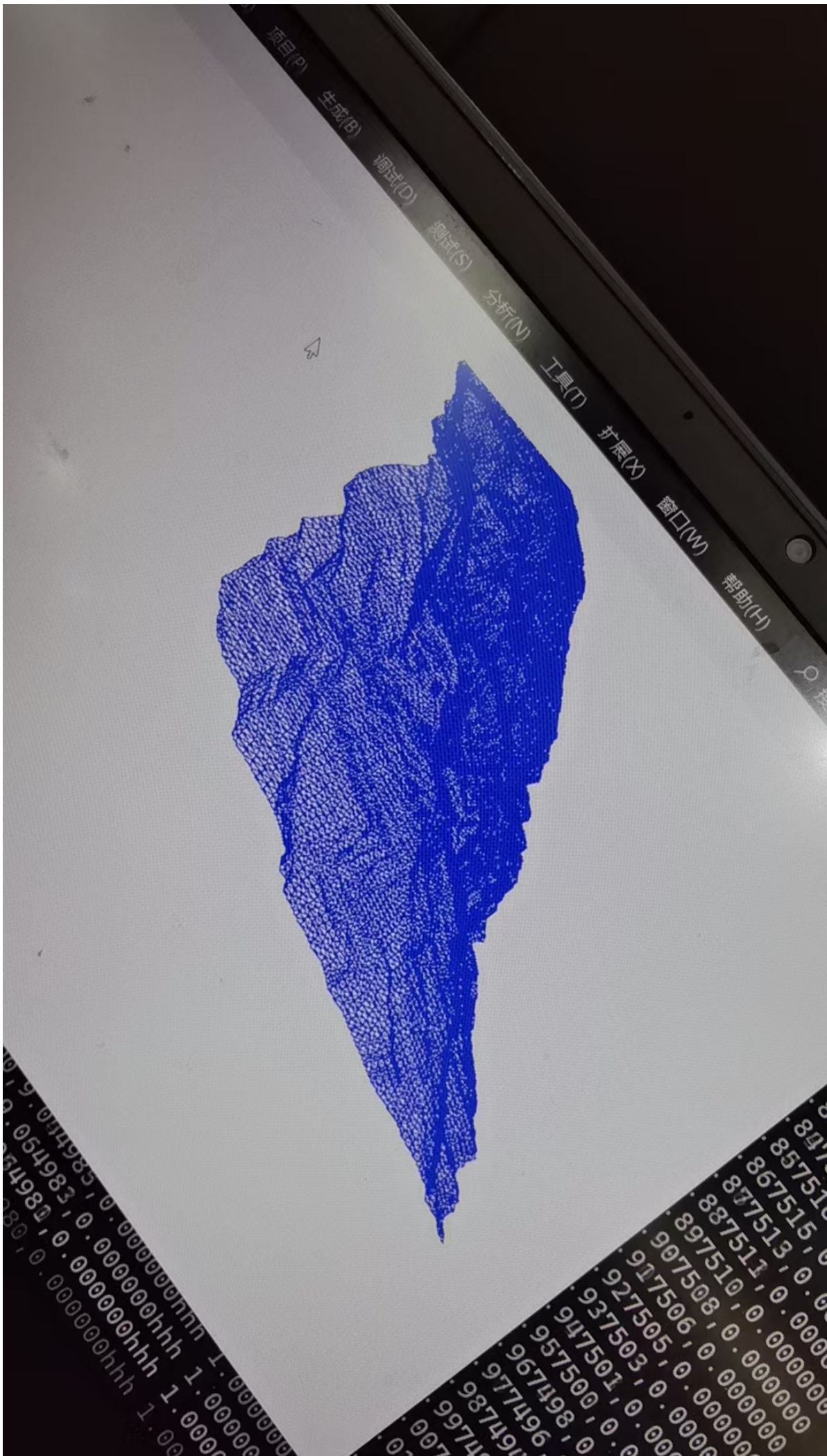
01. bool detect_single_Channel(int i) {
02.     // 检查 x 轴是否重叠
03.     if (ball.min_x > Channel[i].max_x || ball.max_x < Channel[i].min_x) {
04.         return false; // x 轴不重叠, 不相交
05.     }
06.     // 检查 y 轴是否重叠
07.     if (ball.min_y > Channel[i].max_y || ball.max_y < Channel[i].min_y) {
08.         return false; // y 轴不重叠, 不相交
09.     }
10.     // 检查 z 轴是否重叠
11.     if (ball.min_z > Channel[i].max_z || ball.max_z < Channel[i].min_z) {
12.         return false; // z 轴不重叠, 不相交
13.     }
14.     return true; // x、y、z 轴都重叠, 相交
15. }
16. bool detect_channel_ball() {
17.     for (int i = 0; i < Channel.size(); i++) {
18.         if (detect_single_Channel(i) == 1) return 1; // 相交出界
19.     }
20.     return 0;
21. }
22. void clear() {
23.     Channel.clear();
24. }

```

过程中的困难

遇到的大部分困难基本上是方案选择上的纠结以及参数调试。

最初我想着用真实的地形高度图来完成地形的绘制，通过加载不同的高度图也可以实现地图的转换，而且某些游戏（如都市天际线）可以自定义高度图，这样就可以实现地图的自定义。初步的实现效果如下：





可是由于网格数太多，而且地面不是平的，实现碰撞检查会十分复杂（最初我想直接引入开源的物理引擎，如Bullet 物理引擎，并且成功的在项目上实现了构建），不过由于这样的话实现难度骤增，还要去学习物理引擎的使用，和图形学的课程无关，和小组成员讨论后，决定放弃这个想法。

初始时为了实现动态的加载场景同时不影响视觉效果，在参数调试以及方案选择上花费了不少功夫，初始时没有按照一定间隔进行绘制，导致房屋出现了穿模情况，后面进行了改良，隔一定间隔进行绘制，使穿模情况不再发生。其次是小球转弯时，由于房屋是绘制在小球两侧的，因此如果不做预防，小球会撞击房屋，这里刚开始没想着用“清屏”的方法，而是在房屋绘制上下功夫，折腾了好久也没弄出来，遂放弃。采用清屏方式，并且通过重置检查点来实现转向后场景加载也同时到位。实现了预期的效果。